

Incorporating Retrieval Information into the Truncation of Ranking Lists for Better Legal Search

Yixiao Ma

mayx20@mails.tsinghua.edu.cn
DCST, Institute for Internet Judiciary,
Tsinghua University. Quan Cheng
Laboratory
Beijing, China

Qingyao Ai

aiqingyao@gmail.com
School of Computing, University of
Utah
Salt Lake City, Utah, USA

Yueyue Wu

wuyueyue1600@gmail.com
DCST, Institute for Internet Judiciary,
Tsinghua University. Quan Cheng
Laboratory
Beijing, China

Yunqiu Shao

shaoyunqiu14@gmail.com
DCST, Institute for Internet Judiciary,
Tsinghua University. Quan Cheng
Laboratory
Beijing, China

Yiqun Liu*

yiqunliu@tsinghua.edu.cn
DCST, Institute for Internet Judiciary,
Tsinghua University. Quan Cheng
Laboratory
Beijing, China

Min Zhang

z-m@tsinghua.edu.cn
DCST, Institute for Internet Judiciary,
Tsinghua University. Quan Cheng
Laboratory
Beijing, China

Shaoping Ma

mshp@tsinghua.edu.cn
DCST, Institute for Internet Judiciary,
Tsinghua University. Quan Cheng
Laboratory
Beijing, China

ABSTRACT

The truncation of ranking lists predicted by retrieval models is vital to ensure users' search experience. Particularly, in specific vertical domains where documents are usually complicated and extensive (e.g., legal cases), the cost of browsing results is much higher than traditional IR tasks (e.g., Web search) and setting a reasonable cut-off position is quite necessary. While it is straightforward to apply existing result list truncation approaches to legal case retrieval, the effectiveness of these methods is limited because they only focus on simple document statistics and usually fail to capture the context information of documents in the ranking list. These existing efforts also treat result list truncation as an isolated task instead of a component in the entire ranking process, limiting the usage of truncation in practical systems. To tackle these limitations, we propose LeCut, a ranking list truncation model for legal case retrieval. LeCut utilizes contextual features of the retrieval task to capture the semantic-level similarity between documents and decides the best cut-off position with attention mechanisms. We further propose a Joint Optimization of Truncation and Reranking (JOTR) framework based on LeCut to improve the performance of truncation and retrieval tasks simultaneously. Comparison against competitive baselines on public benchmark datasets demonstrates

the effectiveness of LeCut and JOTR. A case study is conducted to visualize the cut-off positions of LeCut and the process of how JOTR improves both retrieval and truncation tasks.

CCS CONCEPTS

• Information systems → Retrieval models and ranking.

KEYWORDS

ranking list truncation, legal case retrieval, efficiency

ACM Reference Format:

Yixiao Ma, Qingyao Ai, Yueyue Wu, Yunqiu Shao, Yiqun Liu, Min Zhang, and Shaoping Ma. 2022. Incorporating Retrieval Information into the Truncation of Ranking Lists for Better Legal Search. In *Proceedings of the 45th Int'l ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '22)*, July 11–15, 2022, Madrid, Spain. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3477495.3531998>

1 INTRODUCTION

Cut-off task aims to truncate ranking lists to balance the effectiveness and efficiency of retrieval results. This benefit is particularly significant in specific domains such as legal case retrieval, where the cost of assessing document relevance is high. Legal case retrieval aims to retrieve relevant legal cases from a corpus of candidate cases regarding a query case. In legal case retrieval, browsing all content in a ranking list takes much more effort than traditional IR tasks (e.g., Web search) mainly for two reasons: Firstly, users tend to examine more search results for legal case retrieval than Web search to find enough relevant information. As shown in Table 1, Shao et al. [25] observed that the number of visited pages per legal search session is over 4.6 times the number of Web search.

*Corresponding author



This work is licensed under a Creative Commons Attribution International 4.0 License.

SIGIR '22, July 11–15, 2022, Madrid, Spain.

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-8732-3/22/07.

<https://doi.org/10.1145/3477495.3531998>

Table 1: A user study on legal case retrieval and Web search. In each session, users could input multiple queries to search for an answer.

Attribute	Legal	Web
task time (s) per session	717.3	94.36
# visited pages per session	18.50	4.000
% clicked pages to total pages per query	0.2000	0.1000
avg. click dwell time (s) per query	38.43	19.62

Also, the percentage of clicked pages to all pages in legal search is twice that of Web search. Secondly, users of legal search make decisions more slowly because of the long length and high reading complexity of legal documents. For example, the average document length of the legal dataset COLIEE [19]2021 is 4843 words, while for MSMARCO [17] it is only 1123. It can be concluded from Table 1 that users spend almost twice the time on a clicked page in legal search than Web search. In summary, the above evidence indicates that legal search involves a much higher search effort than Web search. Therefore, an effective truncation system that removes the irrelevant tail of the result list could potentially reduce legal search users' mental or monetary cost.

Previous truncation approaches [3, 13, 34] are mostly designed for traditional ad-hoc retrieval tasks and only take ranking scores and simple document statistics (e.g., document length and term frequency) as input. While such assumption-free methods can be applied to any type of cut-off task, they are limited by the ignorance of high-level document semantics in their modeling process. In legal case retrieval, the concept of relevance involves complicated semantic relationships between queries and documents. To capture this relationship requires more than ranking scores and document statistics. For example, Arampatzis et al. [1] proposed to find an optimal point to stop reading a ranking list in legal case retrieval based on the distribution of ranking scores. To make the model assumption-free, they designed a normal-exponential mixture to approximate score distributions under all scenes. However, such heuristic distribution functions cannot adapt to datasets with different score distributions, thus producing inferior performance in legal case retrieval.

In this paper, we propose LeCut, a truncation model for legal case retrieval. To address the challenges above, LeCut utilizes hidden features in the retrieval task as additional information. Specifically, LeCut adopts a parallel framework that processes retrieval features to identify both document-wise and list-wise query-document similarities. Then, it uses a recurrent network followed by a Transformer layer with positional embeddings to capture the sequential dependencies in the ranking list. To the best of our knowledge, LeCut is the first to introduce semantic features and context information from the retrieval model to the cut-off task.

In addition, existing efforts treat result list truncation as an isolated task instead of a component in the entire ranking process, which limits the usage of truncation in practical systems. Since the retrieval task and the cut-off task both focus on recognizing relevant documents, the information gather from each could benefit

the other. Therefore, we propose a Joint Optimization of Truncation and Reranking (JOTR) framework, which jointly optimizes the retrieval model and the truncation model by taking their outputs as new features for each other. To adapt the information from LeCut to the legal retrieval model, we propose a mapping function to transform the original output of LeCut into proper cut-off features that could enhance the retrieval task.

To better measure the effect of truncation models on improving the user experience in practice, we tailored and reconstructed an existing cut-off metric based on the user study of legal search. Experimental results on both Canadian and Chinese legal case retrieval datasets using widely-adopted and legal-specific cut-off evaluation metrics demonstrate the effectiveness of LeCut. Comparative experiments are conducted to validate the improvements made by JOTR. Further, we present a case study to visualize the improvement of cut-off predictions made by LeCut and how JOTR improves both tasks simultaneously.

To summarize, our main contributions are:

- We propose LeCut, a ranking list truncation model that novelly incorporates contextual information of the retrieval task into the prediction of optimal cut-off positions.
- Based on LeCut, we propose a Joint Optimization of Truncation and Reranking (JOTR) framework to jointly optimize the retrieval and cut-off tasks.
- Extensive experiments on different cut-off datasets and case studies demonstrate the effectiveness of LeCut and JOTR.

The rest of our paper is organized as follows: Section 2 introduces the design and implementations of LeCut and JOTR. The adopted datasets, evaluation metrics, and experimental setup are described in Section 3. Then, Section 4 elaborates on the experimental results and analysis of our work on multiple datasets. Section 5 reviews the related work. Finally, Section 6 concludes our work and discusses directions for future work. The source code of our work is available at <https://github.com/myx666/LeCut>.

2 METHOD

The overall framework of our proposed method is shown in Figure 1. LeCut takes the ranking list as input to predict a list of cut-off confidence scores $c_1, c_2, \dots, c_l$ representing the probability of truncating the ranking list at position 1, 2, ..., l , respectively. Next, JOTR employs cut-off scores to update the ranking list and optimizes the cut-off and retrieval tasks recurrently. Queries and candidates in the retrieval task are all case documents from legal case retrieval datasets. In the following subsections, we first describe the definition of the cut-off task; then, we elaborate on the implementations of LeCut and JOTR.

2.1 Task Overview

The cut-off task aims to truncate the ranking list at an optimal position in terms of both effectiveness and efficiency. Formally, given a query case q and a list of ranked candidate cases $C = \{case_1, case_2, \dots, case_l\} (l \in \mathbb{N}^+)$, the task is to predict a cut-off position $k \in [1, l]$ that maximises the non-monotonic metrics.

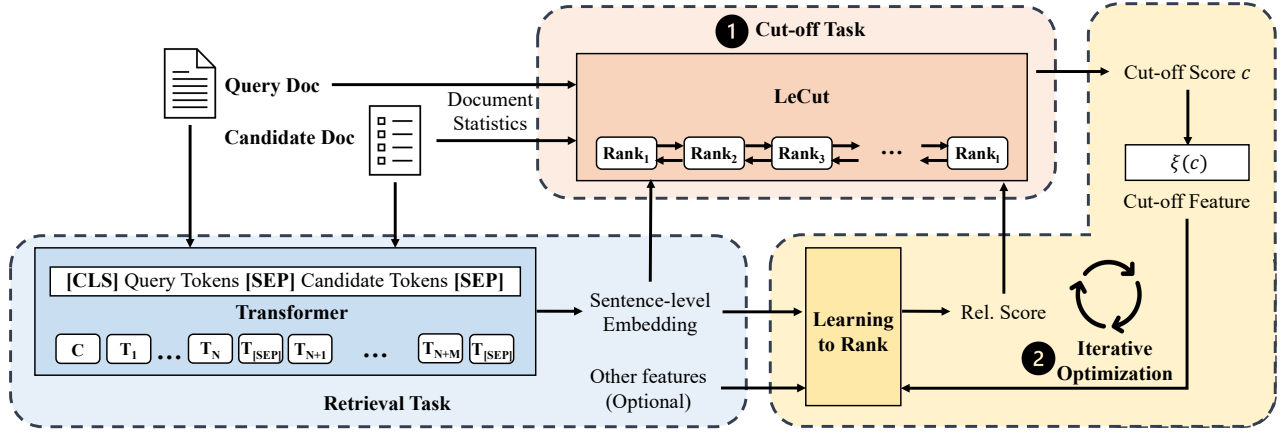


Figure 1: The overall framework of our proposed method. There are two components in the method: 1) The truncation of retrieved ranking lists. 2) JOTR that optimizes the performance of the retrieval and cut-off tasks recurrently. ξ denotes the cut-off score mapping function.

2.2 Feature Preparation

The first step of building a truncation model is to extract input features from query-document pairs. Existing studies [3, 13, 34] on ranking truncation mostly adopt relevance scores and document statistics for cut-off prediction. Although such a lightweight strategy is assumption-free and can be applied to any type of retrieval task, its ignorance of the contextual information in case documents could limit the effectiveness in legal case retrieval. To tackle this problem, we propose to extract three types of input features for LeCut: relevance scores, document statistics, and sentence-level embeddings in the retrieval task.

In terms of document statistics, the document length and tf-idf similarity between neighborhood documents are adopted because they are proved effective by Lien et al. [13]. For the tf-idf similarity, we first regard the candidates of each query as a corpus and compute tf-idf vectors of candidate documents accordingly. Then, the neighborhood tf-idf similarity feature of each candidate document is computed in the following manner:

$$sim_i = \begin{cases} [1, \cos(\text{tf-idf}_1, \text{tf-idf}_2)] & i = 1 \\ [\cos(\text{tf-idf}_{i-1}, \text{tf-idf}_i), \cos(\text{tf-idf}_i, \text{tf-idf}_{i+1})] & i \in [2, l) \\ [\cos(\text{tf-idf}_{l-1}, \text{tf-idf}_l), 1] & i = l \end{cases} \quad (1)$$

where $\cos$ is the cosine similarity function and tf-idf_i is the tf-idf vector of the i -th candidate document. sim_i is a two-dimensional feature representing the lexical similarity of documents to the previous and next documents.

In addition to static document statistics, the interactive information from the pair match of queries and candidates is also beneficial to the cut-off prediction. To this end, sentence-level embeddings from the retrieval task are introduced to our model as an input feature. Specifically, a Transformer in the retrieval task is fine-tuned by the Next Sentence Prediction (NSP) task with retrieval datasets. After tokenization, each query and candidate are concatenated to construct a query-candidate pair and fed into the Transformer. The

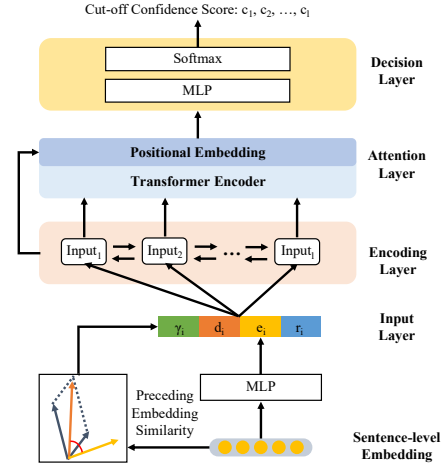


Figure 2: The structure of LeCut. γ_i is the cosine similarity between the sentence-level embedding and its aggregated preceding embeddings, d_i is the document statistics, e_i is the one-dimensional interactive output, and r_i is the relevance score of the i -th document in the ranking list.

sentence-level embedding is the output of the last hidden layer from this Transformer.

2.3 Truncation Model

In this subsection, we introduce the implementation details of LeCut. Figure 2 is the overall structure of LeCut, which is composed of four layers: 1) Input Layer, 2) Encoding Layer, 3) Attention Layer, and 4) Decision Layer. We will detail LeCut in the order of layers.

As discussed in Section 2.2, the raw input of LeCut consists of three parts: relevance score r_i , document statistics d_i , and sentence-level embeddings s_i . Truncation models make a cut-off decision according to list-wise relations in a ranking list, but s_i are sentence-level embeddings representing document-wise similarities in the

retrieval task. In order to incorporate such retrieval information into the cut-off task, s_i needs to be aligned with the list-wise optimization target without losing its similarity information. In consequence, we exploit a parallel framework consisting of two blocks to process s_i list-wise and document-wise separately and combine their results at last. Specifically, the first block is an MLP with a softmax layer that takes s_i as input:

$$e_i = \text{Softmax}(\text{MLP}(s_i)) \quad (2)$$

where e_i denotes the probability of being relevant to the case. In addition to Equation 2, a more important usage of s_i is to represent the correlations in the ranking list. Therefore, the second block computes the embedding similarity between the current document and its preceding documents. In contrast to Wu et al. [34] that only compute the cosine similarity of tf-idf features between neighborhood documents, we pay attention to all candidate documents before the current document in the ranking list because the decision of truncation depends on the relationship between the current document and its precedents. In other words, for the candidate document on the i -th position, we consider the similarity between s_i and $\{s_1, s_2, \dots, s_{i-1}\}$:

$$\gamma_i = \cos(s_i, \sum_{j=1}^{i-1} w_j \cdot s_j), w_j = \frac{\exp(r_j)}{\sum_{k=1}^l \exp(r_k)} \quad (3)$$

Traditional attention mechanism focuses on the contextual connections less when the distance between two cases increases. As for a ranking list, however, documents with higher ranks are supposed to have more attention because of their higher relevance to the query. Therefore, γ_i is represented by the cosine similarity between s_i and the weighted aggregation of its preceding embeddings, and the attention weights depend on their corresponding relevance scores.

Finally, four kinds of features including the relevance score are concatenated as the input to the next layer: $\text{Input}_i = [r_i, e_i, d_i, \gamma_i]$.

Next, the list $\{\text{Input}_1, \text{Input}_2, \dots, \text{Input}_l\}$, as shown in Figure 2, is fed into a Bidirectional LSTM (Bi-LSTM) to exploit sequential dependencies between documents in the ranking list:

$$h = \text{Bi-LSTM}([\text{Input}_1, \text{Input}_2, \dots, \text{Input}_l]) = [h_1, h_2, \dots, h_l] \quad (4)$$

where $\text{Input}_i \in \mathbb{R}^{N \times 6}$ and $h \in \mathbb{R}^{N \times l \times H}$. N is the batch size and H is the hidden size of the Bi-LSTM layer.

After getting the hidden sequential vector h , we leverage a Transformer [30] encoder to represent document embeddings in the ranking list. The multi-head attention mechanism used in the Transformer captures contextual information between documents even at a long distance in the ranking list. As the input of the Transformer encoder, additional positional embeddings p are appended to the hidden embedding h to replace the default sinusoidal positional values in the vanilla Transformer, where $p \in \mathbb{R}^{N \times l \times P}$ are trainable parameters and P is the size of positional embedding for each document. Because the input of a cut-off task is organized as a list, the position information of an input document could be important for the truncation models. Adding positional embeddings helps the model distinguish between documents on different positions. Finally, the generated document embeddings are imported into an

MLP followed by a softmax layer to get the cut-off predictions:

$$m = \text{TransformerEncoder}([h, p]) = [m_1, m_2, \dots, m_l] \quad (5)$$

$$c = \text{Softmax}(\text{MLP}(m)) \quad (6)$$

where m is the document embeddings represented by the Transformer encoder and $c = \{c_1, c_2, \dots, c_l\}$ is a list of cut-off scores. Given a specific ranking truncation metric, c_i denotes the confidence for LeCut to truncate the ranking list at the i -th position. The final prediction of cut-off position is $\text{argmax}(c_1, c_2, \dots, c_l)$.

Suppose $y_1, y_2, \dots, y_l$ are the relevance labels for documents $\text{case}_1, \text{case}_2, \dots, \text{case}_l$, the loss function of LeCut is defined as:

$$\mathcal{L}(y, c) = - \sum_{i=1}^l M(y_1, y_2, \dots, y_i) \log\left(\frac{\exp(c_i/\sqrt{\tau})}{\sum_{j=1}^l \exp(c_j/\sqrt{\tau})}\right) \quad (7)$$

where M is the metric score of the truncated ranking list and τ is a hyper-parameter to control the influence of cut-off confidence scores. M could be any truncation metrics, such as F1 or DCG.

2.4 Joint Optimization of Truncation and Reranking

Previous researchers worked on the retrieval task and cut-off task separately. However, as the subsequent task of the retrieval task, the truncation of ranking lists also relies on the recognition of relevant documents. Thus, it's potentially beneficial to tackle the cut-off and retrieval tasks with one optimization framework. In this paper, we propose JOTR, which works as a loop to jointly train the truncation and retrieval model. In the first iteration of JOTR, the retrieval model predicts a ranking list and corresponding relevance scores r_i for each document in the list. The ranking of documents at this stage depends on r_i entirely without other external information. Then, LeCut takes r_i and related features as the input of the first iteration to predict an initial list of cut-off confidence scores $c = \{c_1, c_2, \dots, c_l\}$. After that, we rerank the previously retrieved results by introducing the cut-off information made by c to the retrieval model.

While the idea of JOTR is straightforward, feeding the truncation model confidence score c to the retrieval model directly could hurt the retrieval performance. One reason for this phenomenon is the distribution disparity between c and the relevance score r . Ideal features for the retrieval model should have high correlations with the document relevance, but the original c_i does not follow this requirement as it is designed to measure the ranking performance of a truncated list with length i . Therefore, we propose a mapping function, ξ , to align the cut-off confidence score with document relevance:

$$\tilde{c}_i = \xi(c_i), i \in [1, l] \quad (8)$$

where $\tilde{c}_i$ is regarded as a cut-off feature tailored for the retrieval task. In this paper, we implement three versions of the mapping function:

① $\xi(c_k) = c_k - c_{k-1}$ is the most straightforward method to represent the relevance feature of the k -th document in the ranking list. Suppose the number of relevant documents among top- i documents in the ranking list is R_i , the F1 score of top- k

documents can be represented as:

$$Precision_k = \frac{R_k}{k}, Recall_k = \frac{R_k}{R_l} \quad (9)$$

$$F1_k = \frac{2Precision_k \times Recall_k}{Precision_k + Recall_k} = \frac{2R_k}{R_l + k} = \frac{2(R_{k-1} + I(y_k))}{R_l + k - 1 + 1} \quad (10)$$

where $I(y_k) = 1$ if the k -th document is relevant to the query else 0. According to Equation 10, $F1_k$ will exceed $F1_{k-1}$ if the k -th document is relevant to the query, and vice versa. As c_k is the confidence score to cut off at position k , the value of c_k is positively correlated with $F1_k$. Therefore, we adopt $c_k - c_{k-1}$ to represent the relevance feature of the k -th document in the ranking list.

② $\xi(c_k) = \frac{c_k}{c_{k-1}}$ and ③ $\xi(c_k) = \ln \frac{c_k}{c_{k-1}}$ use the division of neighborhood confidence scores to represent the relevance feature. The proof is similar to the previous mapping function. Compared to $\frac{c_k}{c_{k-1}}$, $\ln \frac{c_k}{c_{k-1}}$ takes advantage of the sign of the value to indicate the relevance of the k -th document. In Section 4.3, comparative experiments are conducted to illustrate the performances of all mapping functions. Without special notices, we adopt $\xi(c_k) = c_k - c_{k-1}$ as the default mapping function in the rest of this paper.

Finally, we adopt Lightgbm [12], a light but effective learning to rank model, to integrate the relevance score and $\tilde{c}_i$:

$$r'_i = \text{lgb}(r_i, \tilde{c}_i) \quad (11)$$

where lgb is the model of Lightgbm and r'_i is the updated relevance score. We feed the new relevance score r'_i to the training of the truncation model and produce new cut-off confidence scores. The whole process can be carried for multiple rounds. In this way, JOTR can help both the truncation model and the retrieval model better estimate the document relevance with information collected from each other. Also, we design JOTR intentionally with a light function (i.e., lgb) that only considers the relevance scores and confidence scores as the inputs because there is no need to retrain the initial retrieval model. This means JOTR can be applied to any retrieval model efficiently without large computation overhead.

In summary, the algorithm of JOTR is illustrated in Algorithm 1. M_r and M_c represent the evaluation metric scores on the retrieval task and cut-off task in the i -th iteration. The whole algorithm will stop until the improvements of both tasks are below two certain thresholds δ and ϵ at the same time.

3 EXPERIMENTAL SETTINGS

In this section, we describe the setup of our experiments in detail. Specifically, we conduct experiments on six cut-off datasets and adapt a couple of ranking truncation metrics based on the special characteristics of legal search user behavior.

3.1 Datasets

To have a comprehensive evaluation of our method, in this paper, we adopt three retrieval datasets: LeCaRD, CAIL2021-LCR, and COLIEE2020-T1. **Legal Case Retrieval Dataset (LeCaRD)** [15] is the first Chinese legal case retrieval dataset proposed in 2020. **Challenge of AI and Law (CAIL)** [36] is a Chinese legal evaluation of intelligence technology held annually to promote digital justice. CAIL2021-LCR is the dataset for the legal case retrieval

Algorithm 1 JOTR

Input: Relevance scores of the initial ranking list $r = \{r_1, r_2, \dots, r_l\}$; document statistics $d = \{d_1, d_2, \dots, d_l\}$; sentence-level embeddings $s = \{s_1, s_2, \dots, s_l\}$.

Output: A cut-off prediction $q \in [1, l]$; An optimized list of relevance scores r ;

```

1:  $M_r = M'_r = M_c = M'_c = 0$ ;
2:  $c = \{0, 0, \dots, 0\}$ 
3: while  $M'_r - M_r \leq \delta$  or  $(M'_c - M_c \leq \epsilon)$  do
4:    $r' = \text{lgb}(r, \xi(c))$ 
5:    $c' = \text{LeCut}(r', d, s)$ ;
6:    $q = \text{argmax}(c')$ ;
7:    $M_r = M'_r$ ;  $M'_r = \text{Metric}_r(r')$ ;
8:    $M_c = M'_c$ ;  $M'_c = \text{Metric}_c(r', q)$ ;
9:    $r = r'$ ;  $c = c'$ ;
10: end while
11: return  $q, M'_r$ 
```

task in CAIL2021¹. **Competition on Legal Information Extraction / Entailment (COLIEE)** [19] is a competition to develop IR and document entailment methods in the legal domain since 2014. COLIEE2020-T1 is the legal case retrieval dataset of predominantly Federal Court of Canada case laws provided by Compass Law for Task 1 in COLIEE2020. Statistics of three datasets are shown in Table 2. The relevance label of documents are binary in COLIEE2020-T1 and four-class in LeCaRD and CAIL2021-LCR. To unify the evaluation metrics, in this paper, documents labeled 3 or 2 in LeCaRD and CAIL2021-LCR are regarded as relevant cases. Also, for each query in COLIEE2020-T1, we collect top-30 relevant candidates by BM25 [21] as the dataset used in this paper instead of its original size of 200 candidates per query because COLIEE2020-T1 has a much lower proportion of relevant cases than the other two datasets.

In addition, we adopt four competitive neural-based retrieval models to generate ranking lists: BERT [8], RoBERTa [14], refined Lawformer (LFM) [35], and BERT-Crime [39]. BERT and RoBERTa are adopted to generate ranking lists on COLIEE2020-T1 because they are pretrained on English corpus, while LFM and BERT-Crime are adopted to generate ranking lists on LeCaRD and CAIL2021-LCR because they are pretrained on Chinese corpus. All the retrieval models adopt the default hyper-parameter settings in their papers.

The cut-off datasets in this paper are combinations of retrieval datasets and retrieval models. As each of the three retrieval datasets is retrieved by two retrieval models, six cut-off datasets are generated in this paper. The naming of cut-off datasets follows: *Retrieval Dataset Name + Retrieval Model Name*. For example, 'LeCaRD + LFM' refers to the cut-off dataset (i.e., ranking lists) based on LeCaRD and retrieved by a refined LFM.

3.2 Baselines

Our method is evaluated against the following baselines:

- **Fixed-k** truncates the given ranking list at the fixed position k . Fixed truncation is also the most simple and straightforward truncation strategy.

¹<http://cail.cipsc.org.cn/index.html>

Table 2: Statistics of datasets used in this paper. $A(B/C)$ denotes the number of queries in the whole dataset, training set, and test set, respectively.

Dataset	Language	# Documents	# Queries	Doc Length	# Candidates/Query	# Rel. Cases/Query
LeCaRD	Chinese	43,823	107 (87/20)	8275	100	10.33
CAIL2021-LCR	Chinese	50,000	130 (90/40)	2707	100	11.53
COLIEE2020-T1	English	130,000	650 (520/130)	2912	30	5.15

- **Greedy** choose the fixed truncation position k which maximizes the truncation performance on the training set.
- **BiCut** [13] is an assumption-free cut-off approach which represents the non-parametric score distribution of any retrieval model. At each position of the given ranking list, BiCut predicts the probability to continue or cut off.
- **Choppy** [3] is also an assumption-free approach which adopts multi-head attention mechanism to optimize the cut-off task.
- **AttnCut** [34] applies the transformer to capture the context feature within the ranked list for the cut-off task. Besides, AttnCut also employs the reward augmented maximum likelihood (RAML) [18] for a direct optimization. Compared to other previous works, AttnCut has state-of-the-art cut-off performance.
- **Oracle** is the theoretical upper bound of the cut-off task on different metrics.

3.3 Evaluation Metrics

For the retrieval task, we adopt $NDCG@k$ as the evaluation metric, where $k = \{10, 20, 30, 100\}$ on the LeCaRD and CAIL2021-LCR datasets, and $k = \{1, 3, 5, 10\}$ on the COLIEE2020-T1 dataset according to the different numbers of candidate cases per query.

For the cut-off task, we adopt non-monotonic metrics including F1, DCG, and Negative Cumulative Impact (NCI) to reflect both effectiveness and efficiency of the truncation model. Particularly, we follow Wu et al. [34] to adopt the definition of DCG from Järvelin and Kekäläinen [11], which penalizes the impact of irrelevant documents in the truncated ranking list:

$$DCG = \sum_{i=1}^l \frac{2y_i - 1}{\log_2(i + 1)} \quad (12)$$

where l is the length of the truncated ranking list and y_n is the binary label of candidate cases.

NCI is a special truncation metric proposed by Lien et al. [13] which takes into account the effect of negative cases as a linearly increasing utility. The definition of NCI is:

$$NCI = \sum_{i=1}^l \left(\frac{\mathbb{I}(y_i = 1)}{\log_2(i + 1)} - \mathbb{I}(y_i = 0) * \frac{i}{\alpha} \right) \quad (13)$$

where l is the length of the truncated ranking list and α is a hyper-parameter to determine the impact of irrelevant candidate cases. The original α in Lien et al. [13] is set to 10, which is tailored for a Web search dataset. To validate the effectiveness of LeCut in legal case retrieval, we modify the value of α by analyzing the impact of negative cases in both traditional Web search and legal

search. Specifically, we counted the dwell time on landing pages for different domains and labels in the open-source dataset published by Shao et al. [25] and Zhang et al. [38]. As shown in Table 3, users tend to spend more time on the landing page during legal search than Web search, regardless of the relevance of the landing page to their query. Therefore, the negative impact of reading a negative document in legal retrieval should be larger than that in Web search. If we take the median dwell time on the negative landing page to represent the effort (or impact) of reading a page, the ratio (i.e., r_{LW}) of negative penalty in legal retrieval (i.e., $\text{penalty}_{\text{Legal}}$) to Web search (i.e., $\text{penalty}_{\text{Web}}$) should satisfy:

$$r_{LW} = \frac{\text{penalty}_{\text{Legal}}}{\text{penalty}_{\text{Web}}} = \frac{26874}{14728} \approx 1.82 \quad (14)$$

On the other hand, the Web search dataset used in Lien et al. [13] is Robust04 [31], which contains 70 relevant cases in a total number of 300 candidate cases per query. To match the proportion of relevant/irrelevant cases for a reasonable α , NCI in different domains is required to follow:

$$\frac{U_{p\text{Legal}}}{U_{p\text{Web}}} = \frac{U_{n\text{Legal}}}{U_{n\text{Web}}} \quad (15)$$

where U_p and U_n are the positive and negative utility in Equation 13. Finally, given Equation 13, 14, and 15, we have:

$$\frac{\sum_{i=1}^{l_W} \frac{\mathbb{I}(y_i=1)}{\log_2(i+1)}}{\sum_{i=1}^{l_L} \frac{\mathbb{I}(y_i=1)}{\log_2(i+1)}} = \frac{\sum_{i=1}^{l_W} \mathbb{I}(y_i=0) * \frac{i}{\alpha_W}}{\sum_{i=1}^{l_L} \mathbb{I}(y_i=0) * \frac{i}{\alpha_L * r_{LW}}} \quad (16)$$

where l_W and l_L are the length of the truncated ranking list in Web search and legal search, respectively. Suppose that the retrieval model gives relevant cases higher ranks than irrelevant cases, then α in the legal domain can be computed as:

$$\alpha_L = \frac{\sum_{i=1}^{l_{pW}} \frac{1}{\log_2(i+1)} * \sum_{i=l_{pL}}^{l_L - l_{pL}} i}{\sum_{i=1}^{l_{pL}} \frac{1}{\log_2(i+1)} * \sum_{i=l_{pW}}^{l_W - l_{pW}} i} * \frac{\alpha_W}{r_{LW}} \quad (17)$$

where $l_{pW} = 70$ and $l_W = 100$ (cases ranked more than 100 are removed) according to Robust04 [31]. l_{pL} is the number of relevant cases as illustrated in Table 2. For example, $l_L = 100$ and $l_{pL} \approx 10$ in the LeCaRD dataset. Consequently, the reasonable values of α in the legal domain are 221.1, 171.6, and 49.0 for LeCaRD, CAIL2021-LCR, and COLIEE2020-T1, respectively.

3.4 Implementation Details

All models are written in Python. Retrieval models are implemented by HuggingFace Transformer Library [33], and TF-IDF is implemented by Gensim [20] over the whole corpus. Truncation baselines

Table 3: Statistics of dwell time (ms) on landing pages during legal search and Web search. Label indicates whether the landing page is relevant to user’s search intent. 25%, 50%, and 75% are the dwell time at three quartiles, respectively.

Domain	Label	Count	25%	50%	75%
Legal	0	856	13,242.75	26,874	53,119.5
	1	433	28,125	50,021	89,139
Web	0	1,305	6,975	14,728	38,970
	1	2,361	7,614	18,195	54,027

are collected from an open-source website² and hyper-parameters are set to default values. The batch size is 20 for truncation models and 8 for retrieval models. As for LeCut, we set the learning rate as 3×10^{-5} , weight decay as zero, and dropout as zero. The hidden size of the double-layer LSTM inside LeCut and the positional encoding layer is 128 and 32, respectively. For JOTR, the training epoch is 30 and the learning rate is 0.1. All models are trained and evaluated on four 32G NVIDIA V100 GPUs.

4 RESULTS

In this section, we present our experimental results on LeCut and the state-of-the-art baselines for ranking truncation. Specifically, we first compare the performance of all truncation models and then provide ablation studies on the mapping functions of cut-off confidence features and other settings for model training. Finally, we show a case study to illustrate how LeCut and JOTR improve the performance of ranking truncation.

4.1 Comparison against Truncation Baselines

As shown in Table 4, LeCut has the best overall performances on all cut-off datasets compared to traditional methods and other neural-based methods. Most of the improvements achieved by LeCut on six datasets are statistically significant. Although all neural-based methods are trained to optimize F1, LeCut still has competitive results on other metrics. Two exceptions are on the DCG metric, where AttnCut achieves the best results, which indicates that AttnCut is a strong baseline. However, the effectiveness of LeCut is more consistent across all metrics on all datasets. Compared to other neural-based methods, LeCut is not assumption-free and utilizes hidden embeddings from the semantic matching of queries and candidates. Therefore, incorporating intermediate features of the retrieval task into the truncation model significantly improves the truncation of ranking lists. In addition, although Greedy seems to have good results, the performance of such a cut-off method depends on the consistency between the training set and test set entirely. Once the test set has a different distribution of oracle cut-off positions, the results of Greedy will decrease. Lastly, the results of LeCut are lower than oracle results to some extent. There is still space for improvements in the cut-off task.

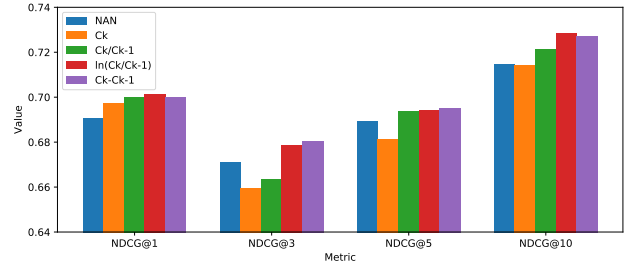


Figure 3: Comparison between different cut-off mapping functions ξ on the 'COLIEE2020-T1+BERT' dataset. c_k is the cut-off confidence score of the k -th document in the ranking list. NAN refers to original ranking list without ξ .

4.2 Performance of JOTR

Similar to Section 4.1, all experiments are conducted on six datasets. The evaluation metrics of the retrieval task on COLIEE2020-T1 differ from the other two retrieval datasets due to the length of ranking lists. The results are shown in Table 5. Note that the result of LFM on LeCaRD is different from Xiao et al. [35], because the authors of LFM adopt a five-cross validation in their paper instead of the official split of training-test set on LeCaRD.

According to Table 5, the retrieval model with cut-off information exceeds its original performance across almost all the metrics on six datasets. Naturally, the optimized ranking list with better NDCG results will have a better cut-off performance using the same truncation model. JOTR is similar to the idea of multi-task learning [22], where both subtasks benefit from sharing the training data and part of the model parameters indirectly. Although the targets of retrieval and cut-off tasks are different, both tasks rely on predicting the score distribution of candidate cases. By transforming the cut-off confidence scores into relevance-related features through an appropriate formula, the iterative optimization process can be regarded as the approximation of two score distributions.

4.3 Ablation Study

To investigate the effectiveness of different blocks used in LeCut, we conduct an ablation study on the 'LeCaRD + LFM' dataset. The experiment does not discuss the utility of traditional document statistics and the attention mechanism already adopted and proved in previous works. As presented in Table 6, the truncation performance decreases without e_i or γ_i feature. When both features are removed from LeCut at the same time, the performance decreases to a larger extent across three metrics. This suggests that e_i and γ_i are both beneficial for predicting an ideal cut-off position. Although both features are generated by the same sentence-level embedding, they play a different role in LeCut. Specifically, the block generating e_i can be regarded as a dimensionality reduction approach to extract similarity between queries and candidates at the semantic level, while the other block generating γ_i focuses on the deviation between the input embedding and the aggregated embeddings at previous positions in the ranking list. Therefore, the information mined by two blocks is supplementary to each other.

²<https://github.com/Woody5962/Ranked-List-Truncation>

Table 4: Evaluation results of the cut-off task on six cut-off datasets. *NAN* refers to results on the original retrieved ranking list without any truncation. *Oracle* is the theoretical upper-bound of the cut-off task. $K = A \setminus B \setminus C$ denotes the fixed truncation strategy at position A , B , and C on the LeCaRD, CAIL2021-LCR, and COLIEE2020-T1 datasets, respectively. $\dagger \setminus \ddagger$ denote significant differences compared to LeCut at a level of p -value $< 0.05 \setminus 0.01$ using Wilcoxon test.

Methods	LeCaRD + BERT-Crime			CAIL2021-LCR + BERT-Crime			COLIEE2020-T1 + BERT		
	F1	DCG	NCI	F1	DCG	NCI	F1	DCG	NCI
NAN	0.3344 $\ddagger$	-7.8013 $\ddagger$	-14.7931 $\ddagger$	0.3456 $\ddagger$	-7.2551 $\ddagger$	-20.5567 $\ddagger$	0.2545 $\ddagger$	-4.7689 $\ddagger$	-6.5406 $\ddagger$
$K = 5 \setminus 5 \setminus 5$	0.3647 $\ddagger$	2.0369	2.4836 $\ddagger$	0.3494 $\ddagger$	2.0529 $\ddagger$	2.4892 $\ddagger$	0.4706 $\ddagger$	0.3686 $\ddagger$	1.3960 $\ddagger$
$K = 10 \setminus 10 \setminus 10$	0.5441 $\ddagger$	2.9376	3.6906 $\ddagger$	0.5509 $\ddagger$	3.1730 $\ddagger$	3.8112 $\ddagger$	0.4166 $\ddagger$	-0.5621 $\ddagger$	1.0980 $\ddagger$
$K = 30 \setminus 30 \setminus 15$	0.7012	3.1376	5.1491	0.7471	4.0833	5.5133	0.3703 $\ddagger$	-1.6034 $\ddagger$	0.1278 $\ddagger$
$K = 50 \setminus 50 \setminus 20$	0.5623 $\ddagger$	0.2057 $\ddagger$	2.2374 $\ddagger$	0.5588 $\ddagger$	0.5718 $\ddagger$	1.0195 $\ddagger$	0.3279 $\ddagger$	-2.7193 $\ddagger$	-1.5110 $\ddagger$
Greedy	0.6864	3.3660	5.1303	0.7305 $\ddagger$	4.1189	5.4758	0.4355 $\ddagger$	0.6335	1.4053 $\ddagger$
BiCut [13]	0.6181 $\ddagger$	3.0729	4.3544 $\ddagger$	0.6124 $\ddagger$	2.8760 $\ddagger$	2.6878 $\ddagger$	0.4627 $\ddagger$	0.1947 $\ddagger$	1.3324 $\ddagger$
Choppy [3]	0.6782 $\ddagger$	3.3660	4.9118 $\ddagger$	0.7789 $\ddagger$	4.9928	5.6934 $\ddagger$	0.5263 $\ddagger$	0.6124 $\ddagger$	1.5032 $\ddagger$
AttnCut [34]	0.6701	3.0677	4.5496	0.8061	5.0812	5.8866	0.5350 $\ddagger$	0.6172 $\ddagger$	1.5040 $\ddagger$
LeCut	0.7043	3.4525	5.1585	0.8126	5.1617	5.9377	0.5525	0.7513	1.5627
Oracle	0.8039	4.6590	5.7675	0.8842	5.9110	6.4576	0.7059	1.4696	1.8067

Methods	LeCaRD + LFM			CAIL2021-LCR + LFM			COLIEE2020-T1 + RoBERTa		
	F1	DCG	NCI	F1	DCG	NCI	F1	DCG	NCI
NAN	0.3344 $\ddagger$	-8.2269 $\ddagger$	-14.7461 $\ddagger$	0.3456 $\ddagger$	-7.2179 $\ddagger$	-20.5475 $\ddagger$	0.2545 $\ddagger$	-4.7226 $\ddagger$	-6.5369 $\ddagger$
$K = 5 \setminus 5 \setminus 5$	0.3390 $\ddagger$	1.8743	2.3248 $\ddagger$	0.3849 $\ddagger$	2.3153 $\ddagger$	2.6236 $\ddagger$	0.4724 $\ddagger$	0.3953 $\ddagger$	1.4182
$K = 10 \setminus 10 \setminus 10$	0.5028 $\ddagger$	2.3628	3.3831 $\ddagger$	0.5376 $\ddagger$	3.1572 $\ddagger$	3.7838 $\ddagger$	0.4301 $\ddagger$	-0.4562 $\ddagger$	1.1629 $\ddagger$
$K = 30 \setminus 30 \setminus 15$	0.6501	2.1907	4.6139	0.7092	3.6760	5.1963	0.3733 $\ddagger$	-1.5828 $\ddagger$	0.1332 $\ddagger$
$K = 50 \setminus 50 \setminus 20$	0.5456 $\ddagger$	-0.4382 $\ddagger$	1.9937 $\ddagger$	0.5708 $\ddagger$	0.7331 $\ddagger$	1.2946 $\ddagger$	0.3245 $\ddagger$	-2.6933 $\ddagger$	-1.5384 $\ddagger$
Greedy	0.6367	2.4365	4.5955	0.7096	3.6810	5.1969	0.4651 $\ddagger$	0.7428	1.4334 $\ddagger$
BiCut [13]	0.5596 $\ddagger$	2.2788	3.4033 $\ddagger$	0.6207 $\ddagger$	3.2005 $\ddagger$	3.9413 $\ddagger$	0.4717 $\ddagger$	0.1316 $\ddagger$	1.3860 $\ddagger$
Choppy [3]	0.6367 $\ddagger$	2.3756 $\ddagger$	4.5655 $\ddagger$	0.7116 $\ddagger$	3.7978 $\ddagger$	5.2778 $\ddagger$	0.5469	0.7572	1.5281
AttnCut [34]	0.6157	2.6619	4.3685 $\ddagger$	0.7190	4.3603	5.3232	0.5480	0.8689	1.5413
LeCut	0.6624	2.6320	4.7544	0.7428	4.3915	5.6004	0.5592	0.7868	1.5561
Oracle	0.7574	4.0279	5.2489	0.8542	5.5512	6.1213	0.7145	1.5291	1.8482

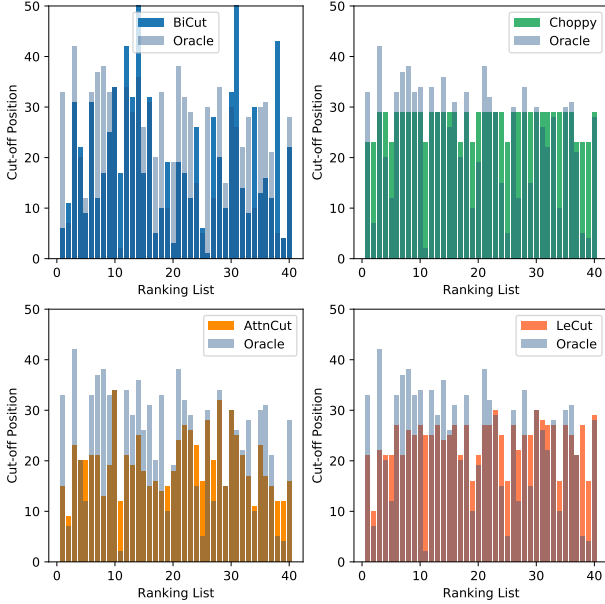
Table 5: Results of retrieval and cut-off tasks with or without JOTR.

Dataset	Method	NDCG@10	NDCG@20	NDCG@30	NDCG@100	F1	DCG	NCI
LeCaRD + LFM	LeCut	0.7654	0.7623	0.8057	0.8910	0.6624	2.6320	4.7544
	LeCut + JOTR	0.7735	0.7850	0.8186	0.8988	0.6793	2.7288	4.8862
LeCaRD + BERT-Crime	LeCut	0.8352	0.8315	0.8674	0.9230	0.7043	3.4525	5.1585
	LeCut + JOTR	0.8367	0.8397	0.8789	0.9262	0.7188	3.5851	5.3146
CAIL2021-LCR + LFM	LeCut	0.8468	0.8443	0.8615	0.9268	0.7428	4.3915	5.6004
	LeCut + JOTR	0.8974	0.8767	0.8817	0.9423	0.7574	4.7557	5.6188
CAIL2021-LCR + BERT	LeCut	0.8663	0.8653	0.8822	0.9176	0.8126	5.1617	5.9377
	LeCut + JOTR	0.8819	0.8817	0.8997	0.9314	0.8217	5.2271	6.1549

Dataset	Method	NDCG@ 1	NDCG@ 3	NDCG@ 5	NDCG@ 10	F1	DCG	NCI
COLIEE2020-T1 + BERT	LeCut	0.6904	0.6709	0.6894	0.7146	0.5525	0.7513	1.5627
	LeCut + JOTR	0.7000	0.6803	0.6950	0.7269	0.5668	0.7332	1.5846
COLIEE2020-T1 + RoBERTa	LeCut	0.6852	0.6813	0.6938	0.7268	0.5592	0.7868	1.5561
	LeCut + JOTR	0.6923	0.6889	0.6946	0.7324	0.5635	0.7119	1.5818

Table 6: Ablation study of LeCut on the 'LeCaRD + LFM' dataset.

Model	F1	DCG	NCI
LeCut	0.6624	2.6320	4.7544
w/o e_i	0.6547 (↓ 1.2%)	2.4812 (↓ 5.7%)	4.6912 (↓ 1.3%)
w/o γ_i	0.6560 (↓ 1.0%)	2.4299 (↓ 7.7%)	4.6710 (↓ 1.8%)
w/o $e_i + \gamma_i$	0.6475 (↓ 2.2%)	2.4162 (↓ 8.2%)	4.6570 (↓ 2.0%)

**Figure 4: The distribution of cut-off positions on the 'CAIL2021-LCR + LFM' dataset. X-axis represents different ranking lists and Y-axis represents positions for cutting off the ranking lists. In each figure, the distribution of cut-off positions predicted by a truncation model is compared to the oracle cut-off positions.**

We also compare the effectiveness of different mapping functions introduced in Section 2.4, and the results are shown in Figure 3. Except for the original cut-off confidence score c_k , all the other cut-off features improve the performance of the retrieval task compared to the original retrieval model. Additionally, both $\ln \frac{c_k}{c_{k-1}}$ and $c_k - c_{k-1}$ have competitive results. In practice, we adopt $c_k - c_{k-1}$ as our mapping function. Also, it can be observed from Figure 3 that adding c_k even decreases the results of retrieval, because the distribution of original c_k and relevance scores do not match. In contrast, $\xi(c_k)$ is beneficial to retrieving relevant candidates. This phenomenon illustrates the possibility of utilizing latent information inside the cut-off task to improve the retrieval task.

4.4 Case Study

Towards an in-depth understanding of the effectiveness of LeCut and JOTR, we perform further analysis on some specific cases. The first case study, as shown in Figure 4, is to present the differences

Iteration 0

Rank	1	2	3	4	5	6	7	8	9	10	11	12	...	21
Doc ID	83736	84638	99872	57037	86583	95982	57135	80362	61906	36268	59315	41721	...	31638
Label	1	1	1	1	0	0	0	0	0	0	1	0	...	1
Rel Score	1.93	1.19	1.19	1.19	1.19	1.19	1.19	1.19	0.78	0.55	0.55	0.55	...	0.43
Cut-off Feature	77.08	9.26	8.06	4.87	4.83	0.25	0.24	1.19	0.33	1.41	2.93	-0.04	...	2.81

Update Ranking

Iteration 1

Rank	1	2	3	4	5	6	7	8	9	10	11	12	...	21
Doc ID	99872	83736	57037	86583	84638	59315	31638	14068	80362	95897	24863	36268	...	...
Label	1	1	1	0	1	1	1	0	0	0	0	0	...	...
Rel Score	3.17	3.05	2.83	2.83	2.79	1.50	1.36	0.97	0.89	0.83	0.74	0.68	...	...
Cut-off Feature	77.42	12.82	8.01	1.05	4.79	4.9	-0.48	2.53	4.46	1.86	-1.55	1.12	...	...

Update Ranking

Figure 5: An example of JOTR. The ranking list belongs to query 53030 in the 'CAIL2021-LCR + LFM' dataset. Rel Score is the relevance score of each candidate document predicted by the retrieval model. Cut-off Feature is the output of the cut-off score mapping function.

of cut-off positions between LeCut and other competitive baselines on the test set of 'CAIL2021-LCR + LFM'. Of all methods presented in Figure 4, LeCut resembles the oracle cut-off strategy best. By comparison, BiCut has the most obvious difference against oracle in terms of cut-off positions. For some cases, BiCut even makes extreme cut-off predictions (either too short or too long). Additionally, although Choppy has competitive cut-off results, most of its cut-off predictions are in the range of [20, 30]. It is challenging for Choppy to truncate ranking lists to a more diverse extent with merely the information of ranking scores.

Figure 5 describes how JOTR improves the retrieval task and cut-off task sequentially. *Iteration 0* is the initial ranking list retrieved by LFM regarding query 53030 in the 'CAIL2021-LCR + LFM' dataset. Then, LeCut takes the ranking information, including relevance scores and other features, as the input to predict a cut-off confidence score for each position. Finally, the mapping function converts cut-off confidence scores into cut-off features for retrieval in the next iteration. As shown in Figure 5, relevant cases 59315 and 31638 both have lower rankings compared to other relevant cases in *Iteration 0*. However, the cut-off stage gives these two cases higher cut-off features (i.e., 2.93 and 2.81) than irrelevant cases next to them. Consequently, the retrieval model at the next iteration gives both cases higher rankings considering their prominent cut-off features. Meanwhile, an irrelevant case 86583 which is ranked in the fourth place during *Iteration 1* will be moved to a lower place in *Iteration 2* because of its relatively low cut-off features (i.e., 1.05). In summary, the details of our algorithm described above demonstrate the practicability of improving the retrieval task and cut-off task through sharing features at the decision layer recurrently.

5 RELATED WORK

The idea of cutting off ranking lists to promote search efficiency was proposed in 1981 [29]. Identifying the optimal cut-off positions not only improves the non-monotonic metric such as F1 [3, 13, 34] but also serves as a prerequisite for retrieval systems with multiple stages [7, 16] because in such systems, the highly-effective model at

the second stage is usually computationally expensive. In addition, the cut-off task is also utilized to comprehend the prediction belief of retrieval models beyond the scope of a single relevance score [5]. In some professional domains requiring additional expertise, such as the legal or patent domain, the truncation of retrieval results significantly reduces users' search effort [1, 6, 25].

As a specific Information Retrieval (IR) task, Legal case retrieval is essential to ensure legal justice. Relevant preceding cases can be crucial references or even directly involved in the decision-making process of a judgment [10]. Therefore, legal case retrieval has received growing attention in recent years [4, 28]. So far, there are mainly two categories of legal case retrieval models: Expert Knowledge-based models [23, 37] and Natural Language Processing (NLP) models. Recent effective models [9, 26, 27] are mainly based on NLP models. As a representative technique, models based on pre-trained language models (PLMs) [2, 24, 32, 39] prevail on various legal tasks, including legal case retrieval. In this paper, we adopt hidden features from PLMs-based approaches for the cut-off task.

6 CONCLUSION

This paper proposes LeCut, a ranking list truncation model which employs hidden features from the retrieval task to capture the semantic-level similarity between documents. In addition to training the truncation model in isolation, JOTR is proposed to optimize both the retrieval and cut-off task simultaneously. Experimental results on benchmark datasets demonstrate the effectiveness of LeCut and JOTR. A case study is conducted to visualize the cut-off positions of LeCut and how JOTR improves both tasks.

On the other hand, our proposed work is generic and adaptive to other domains that require the efficiency of retrieval results. In the future, we will discover the application of LeCut and JOTR in other domains.

7 ACKNOWLEDGEMENTS

This work is supported by the National Key Research and Development Program of China (2018YFC0831700), Natural Science Foundation of China (Grant No. 61732008, 62002194), and Tsinghua University Guoqiang Research Institute.

REFERENCES

- [1] Avi Arampatzis, Jaap Kamps, and Stephen Robertson. 2009. Where to stop reading a ranked list? Threshold optimization using truncated score distributions. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 524–531.
- [2] Arian Askari and Suzan Verberne. 2021. Combining lexical and neural retrieval with longformer-based summarization for effective case law retrieval. (2021).
- [3] Dara Bahri, Yi Tay, Che Zheng, Donald Metzler, and Andrew Tomkins. 2020. Choppy: Cut transformer for ranked list truncation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1513–1516.
- [4] Trevor Bench-Capon, Michal Araszkievicz, Kevin Ashley, Katie Atkinson, Floris Bex, Filipe Borges, Daniele Bourcier, Paul Bourguine, Jack G Conrad, Enrico Francesconi, et al. 2012. A history of AI and Law in 50 papers: 25 years of the international conference on AI and Law. *Artificial Intelligence and Law* 20, 3 (2012), 215–319.
- [5] Daniel Cohen, Bhaskar Mitra, Oleg Lesota, Navid Rekabsaz, and Carsten Eickhoff. 2021. Not All Relevance Scores are Equal: Efficient Uncertainty and Calibration Modeling for Deep Retrieval Models. *arXiv preprint arXiv:2105.04651* (2021).
- [6] Gordon V Cormack and Mona Mojdeh. 2009. Machine Learning for Information Retrieval: TREC 2009 Web, Relevance Feedback and Legal Tracks.. In *TREC*.
- [7] J Shane Culpepper, Charles LA Clarke, and Jimmy Lin. 2016. Dynamic cutoff prediction in multi-stage retrieval systems. In *Proceedings of the 21st Australasian Document Computing Symposium*. 17–24.
- [8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [9] Filippo Galgani, Paul Compton, and Achim Hoffmann. 2012. Citation based summarisation of legal texts. In *Pacific Rim International Conference on Artificial Intelligence*. Springer, 40–52.
- [10] Hanjo Hamann. 2019. The German Federal Courts Dataset 1950–2019: From Paper Archives to Linked Open Data. *Journal of Empirical Legal Studies* 16, 3 (2019), 671–688.
- [11] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems (TOIS)* 20, 4 (2002), 422–446.
- [12] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems* 30 (2017), 3146–3154.
- [13] Yen-Chieh Lien, Daniel Cohen, and W Bruce Croft. 2019. An assumption-free approach to the dynamic truncation of ranked lists. In *Proceedings of the 2019 ACM SIGIR International Conference on Theory of Information Retrieval*. 79–82.
- [14] Yinhao Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [15] Yixiao Ma, Yunqiu Shao, Yueyue Wu, Yiqun Liu, Ruizhe Zhang, Min Zhang, and Shaoping Ma. 2021. LeCaRD: A Legal Case Retrieval Dataset for Chinese Law System. *Information Retrieval (IR)* 2 (2021), 22.
- [16] Hafeezul Rahman Mohammad, Keyang Xu, Jamie Callan, and J Shane Culpepper. 2018. Dynamic shard cutoff prediction for selective search. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. 85–94.
- [17] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A human generated machine reading comprehension dataset. In *CoCo@ NIPS*.
- [18] Mohammad Norouzi, Samy Bengio, Navdeep Jaitly, Mike Schuster, Yonghui Wu, Dale Schuurmans, et al. 2016. Reward augmented maximum likelihood for neural structured prediction. *Advances In Neural Information Processing Systems* 29 (2016), 1723–1731.
- [19] Juliano Rabelo, Mi-Young Kim, Randy Goebel, Masaharu Yoshioka, Yoshinobu Kano, and Ken Satoh. 2020. COLIEE 2020: methods for legal document retrieval and entailment. In *JSAIL International Symposium on Artificial Intelligence*. Springer, 196–210.
- [20] Radim Řehůřek, Petr Sojka, et al. 2011. Gensim—statistical semantics in python. Retrieved from *gensim.org* (2011).
- [21] Stephen E Robertson, Steve Walker, Susan Jones, Micheline M Hancock-Beaulieu, Mike Gatford, et al. 1995. Okapi at TREC-3. *Nist Special Publication Sp* 109 (1995), 109.
- [22] Sebastian Ruder. 2017. An overview of multi-task learning in deep neural networks. *arXiv preprint arXiv:1706.05098* (2017).
- [23] Manavalan Saravanan, Balaraman Ravindran, and Shivani Raman. 2009. Improving legal information retrieval using an ontological framework. *Artificial Intelligence and Law* 17, 2 (2009), 101–124.
- [24] Yunqiu Shao, Jiaxin Mao, Yiqun Liu, Weizhi Ma, Ken Satoh, Min Zhang, and Shaoping Ma. 2020. BERT-PLI: Modeling Paragraph-Level Interactions for Legal Case Retrieval.. In *IJCAI*. 3501–3507.
- [25] Yunqiu Shao, Yueyue Wu, Yiqun Liu, Jiaxin Mao, Min Zhang, and Shaoping Ma. 2021. Investigating User Behavior in Legal Case Retrieval. (2021).
- [26] Vu Tran, Minh Le Nguyen, Satoshi Tojo, and Ken Satoh. 2020. Encoded summarization: summarizing documents into continuous vector space for legal case retrieval. *Artificial Intelligence and Law* 28, 4 (2020), 441–467.
- [27] Vu Tran, Minh Le Nguyen, and Ken Satoh. 2019. Building legal case retrieval systems with lexical matching and summarization using a pre-trained phrase scoring model. In *Proceedings of the Seventeenth International Conference on Artificial Intelligence and Law*. 275–282.
- [28] Marc Van Opijnen and Cristiana Santos. 2017. On the concept of relevance in legal information retrieval. *Artificial Intelligence and Law* 25, 1 (2017), 65–87.
- [29] Cornelis J van Rijsbergen. 1981. Retrieval effectiveness. *Information retrieval experiment* (1981), 32–43.
- [30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*. 5998–6008.
- [31] Ellen M Voorhees et al. 2004. Overview of TREC 2004.. In *Trec*.
- [32] Hannes Westermann, Jaromir Savelka, and Karim Benyekhlef. 2020. Paragraph similarity scoring and fine-tuned BERT for legal information retrieval and entailment. In *JSAIL International Symposium on Artificial Intelligence*. Springer, 269–285.
- [33] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al.

2019. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771* (2019).
- [34] Chen Wu, Ruqing Zhang, Jiafeng Guo, Yixing Fan, Yanyan Lan, and Xueqi Cheng. 2021. Learning to Truncate Ranked Lists for Information Retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 4453–4461.
- [35] Chaojun Xiao, Xueyu Hu, Zhiyuan Liu, Cunchao Tu, and Maosong Sun. 2021. Lawformer: A Pre-trained Language Model for Chinese Legal Long Documents. *AI Open* (2021).
- [36] Chaojun Xiao, Haoxi Zhong, Zhipeng Guo, Cunchao Tu, Zhiyuan Liu, Maosong Sun, Yansong Feng, Xianpei Han, Zhen Hu, Heng Wang, et al. 2018. Cail2018: A large-scale legal dataset for judgment prediction. *arXiv preprint arXiv:1807.02478* (2018).
- [37] Yiming Zeng, Ruili Wang, John Zeleznikow, and Elizabeth Kemp. 2005. Knowledge representation for the intelligent legal case retrieval. In *International Conference on Knowledge-Based and Intelligent Information and Engineering Systems*. Springer, 339–345.
- [38] Fan Zhang, Jiaxin Mao, Yiqun Liu, Weizhi Ma, Min Zhang, and Shaoping Ma. 2020. Cascade or recency: Constructing better evaluation metrics for session search. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 389–398.
- [39] Haoxi Zhong, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. 2019. *Open Chinese Language Pre-trained Model Zoo*. Technical Report. <https://github.com/thunlp/openclap>